Storage Classes in C

UNIT-3

Storage classes in C are used to describe the scope, visibility and lifetime of a variable during the runtime of a program.

The Scope of a variable determines over what region of the program a variable is available for use.

Visibility refers to the accesibility of a variable from memory.

The lifetime/longevity of a variable refers to the period of time in which the variable is available in the memory and retains a given value during execution of a program.

C defines four scopes for an identifier/variable. They are as follows:

File Scope:

Starts at the beginning of the file and ends with the end of the file.

Variables that have file scope are global. All static variables are comes under file scope.

Block Scope:

Begins with the opening bracket and ends with closing bracket. { }

Variables within block scope are local to their block.

Function Scope:

Begins with the opening "{" of a function and ends with closing "}" of the function.

Function parameters are included in function & block scope.

Program Scope:

Variables declared outside of a function is said to be program scope. These variables are available throughout the program.

Scanned by CamScanner

Example :

```
#include<stdio.h>
int x = 0;
float y = 1.0;
void function1()
{
  ==
}
static int z = 0;
static void function2()
{
  ==
}
void main()
{
    int i = 30;
    printf("within outer block i = %d", i);
    {
        int i = 10;
        printf("within inner block i = %d", i);
        i++;
    }
    printf("within outer block i = %d", i);
}
```

Program Scope

File Scope

function scope

block scope

C Language provides four storage classes :

(1) auto
(2) extern
(3) static
(4) register

Scanned by CamScanner

1. auto :

This is the default storage class for all variables declared inside a function or a block.

These variables are created when a function is called and destroyed when function exited.

A variable declared inside a function ~~which~~ without storage class specification is by default auto variable.

Any auto variable local to the main will be alive throughout the whole program although it is active only in main.

Example :

```
#include<stdio.h>
void fun1();
void fun2();
int main()
{   int m=100;
    fun2()
    printf("%d", m);
  return 0;
}
void fun1()
{   int m=10;
    printf("\n %d", m);
}
void fun2()
{     int m=100;
      fun1()
      printf("%d \n", m);
}
```

Scanned by CamScanner

2. Extern :

Variables that are both alive and active throughout the entire program are known as external variables.

External variables are declared outside all functions.

Once a variable has been declared as global any function can use it and change its value.

Example :

```
#include <stdio.h>
int fun1();
int fun2();
int main()
{
    extern int y;
      y = 10;
      printf("y = %d", y);
      printf("y = %d", fun1());
      printf("y = %d", fun2());
     return 0;
}
int y;
int fun1()
{
     y = 50;
    return y;
}
int fun2()
{  int y;
     y = 100;
} return y;
```

Scanned by CamScanner

"extern" declaration tells the Compiler that the variable is defined somewhere in the program.

"extern" declaration doesnot allocate storage space for variables.

"extern" modifier is most Commonly used when there are two or more files belongs to the same program sharing the same global variables or functions.

Example:

```
// main.c
#include <stdio.h>
int ccount;
extern void fun1();
int main()
{
    ccount = 5;
    fun1();
  return 0;
}
```

```
// example.c
#include<stdio.h>
 extern int ccount;
 void fun1()
 { printf("ccount is %d", count);
 }
```

Compile these files as follows:

```
C:\> gcc main.c example.c -o output ↵
C:\> output.exe ↵
```

Output : ccount is 5

Scanned by CamScanner

3. Static :

A static variable can be declared using static keyword. The static variable persists untill the end of program.

A static variable may be either internal or external. Internal static variables are declared inside a function.

The internal static variables are like auto variables except that they will remain in memory throughout the program.

The static variable is initialized only once during compilation. By default static variables are initialized to zero.

An external static variable is declared outside all functions and is available to all the functions in the file.

The difference between static external variable and simple external variable is that static variable is available only within the file, while simple external variable can be accessed by other files in the program.

If we declare a function with static storage class then that function will be only accessible in that file only.

Example :

```
#include<stdio.h>
void fun();
static int count = 5; // external
int main()
{   int i;
    for(i=0; i<3; i++)
        fun();
return 0;
}
void fun()
{
  static int j = 5; // internal
  j--; count++;
printf("j= %d, count= %d",
          j, count)
}
```

Scanned by CamScanner

4. register :

The register variables are created inside a machines register (inside CPU) instead of main memory.

As registers are faster than memory, frequently accessible variables are declared using register for faster execution of program.

Syntax : register int count;

register is a request to the compiler and if registers are not available, C compiler will convert register variable into auto variable.

Summary:

Storage class	Where declared	Storage area	Initial value	Scope	Lifetime
auto	inside a function or block	stack	Garbage	within block or function	end of block or function
extern	outside all the functions	Data Segment	Zero	Global, multiple files	Entire program
static	Inside a function or outside of all functions	Data segment	Zero	within the declared block/ only in the file	Till the end of program
register	Inside a function or block	CPU register	Garbage	only in the function or block	end of block or function

Scanned by CamScanner

Functions in C

A function is a self-contained block of code that performs a particular task.

Function facilitates top-down moduler programming. In this programming style, the high level logic of the overall problem is solved first and then low level functions are addressed later.

Function reduce the size and developement time of programs.

Function provides easy debugging. That is using function it is easy to locate error in program.

C functions can be divided into two categories: library functions and user defined functions.

Library functions comes with C compilers, so no need to write code for these functions.

Example printf(), scanf(), sqrt(), pow() etc.

User defined functions has to be developed by the user at the time of writing program.

Example main(), add(), factorial() etc.

Header file and Library:

Header files tell the compiler how to call some functionality without knowing how they works.

Header files contains function declarations/prototypes.

They also contains datatype and constants used within the libraries.

We use "#include" to use header files in program.

Scanned by CamScanner

Library is the place where the actual functionality is implimented. i.e ~~that~~ they contain function body.

Libraries contains object code and these object codes are linked with user program and they become the part of the executable code.

These libraries are specifically used at compile time which means the library should be present in correct location where compile can found during compilation.

Example: math.h is a header file which includes the prototype for functions calls like sqrt(), pow() etc. where as libm.lib, libmmt.lib are some math libraries.

header files

- They have the extension ".h".
- They contain function declarations
- They are available in "include" sub directory
- Header files are are human readable since they are in the form of source code.
- Header files in our program are included by using a command #include which is internally handled by pre-processor.

Library files

- They have the extension ".lib".
- They contain function definitions
- They are available in "lib" sub directory
- Library files are not human readable since they are in ~~machine~~ the form of machine code.
- Library files in our program are included in last stage by a special module called linker.

Scanned by CamScanner

Elements of user defined functions :

There are three elements related to functions.

1. function definition
2. Function call.
3. function declaration.

The function definition is an independent program module that is specifically written to implement the requirement of the function.

In order to use the function we need to invoke it in the required place of the program. This is called function call.

The function should be declared at the beginning of the program so that it can be called in different place of the program.

1. Function definition :

A function definition includes the following elements.

- Function name.
- Return type
- List of parameters.
- function statements.
- Local variable declaration
- Return statement.

Scanned by CamScanner

The general form of a function definition is:

```
function_return_type function_name (parameter list)
{
    local variable declaration;
    executable statements;
    ==
    return statement;
}
```

Return type specifies the type of value the function will return to the calling program. If function not returning anything we need to specify return type as "void".

The function name is any valid identifier. So we must follow the rules of variable name for function name.

The parameter list declares the variables that will receive the data sent by the calling program. The parameters are also known as arguments.

The parameter list contains declaration of variables separated by commas and surrounded by parenthesis.

When a function doesnot receive any value we can represent it by using void keyword. Many compilers accepts empty set of parenthesis with-out specifying anything.

The function body contains the declarations and statements necessary for performing the required task.

Scanned by CamScanner

The body enclosed in braces, contains three parts:

1. local variable declaration
2. Function statements to perform task
3. A return statement that returns the value evaluated by the function.

If a function doesnot return any value we can ~~omit~~ omit the return statement.

Examples:

(1)
```
float multiply (float x, float y)
{
    float result;
    result = x * y;
    return result;
}
```

(2)
```
void addition (int a, int b)
{
    printf ("Sum = %d", a+b)
    return ; // optional
}
```

(3)
```
void display (void)
{
    printf ("No type, no parameters");
}
```

In C, a function can return only one value per call. The return statement can take one of the following form:

```
return; // doesnot return any value
   or
return (expression);
```

Scanned by CamScanner

A function may have more than one return statement. This situation arises when the value returned is based on certain conditions.

```
if (x <= 0)
    return (0);
else
    return 1;
```

When a value is returned it is automatically type cast into function return type.

```
int product()
{
    return (2.5 * 3.0);
}
```

This will return value 7.

2. Function Calls :

A function can be called using the function name followed by a list of actual parameters, if any, in the parenthesis.

```
void main()
{ int y;
  y = multiply (10, 5);
    printf ("%d\n", y);
}
```

When the compiler encounters a function call, the control is transfered to the function multiply().

After execution of multiply() value is returned to the main() function.

Scanned by CamScanner

If the actual parameters are more than the formal parameters the extra arguments will be discarded.
If the actual parameters are less than the formals the unmatched formal arguments will be initialized with garbage value.
Any mismatch in datatype may also result in some unexpected results.

3. Function declaration :

Function declaration is also known as function prototype.
Like variables, all functions in c must be declared before they are used.
A function declaration contains the following elements.

function return type function_name (parameter list);

Example: int multiply (int, int);

A prototype declaration may be placed in two places.
  - Above all the functions. // Global prototype
  - Inside a function definition. // local prototype
In Global prototype function declaration is placed above all the functions. Such declaration are available for all the functions in the program.
When we place declaration of a function inside definition of other function, this is called local declaration.
It is a good programming style to declare prototypes in Global declaration section before main().

Scanned by CamScanner

Category of functions :

Based on arguments and return type of functions, they can be classified as the following categories:

1. With No argument and No. return value.
2. With arguments and no return value.
3. With arguments and one return value
4. With no-arguments and but one return value.
5. Function returning multiple values.

1. Functions with no argument and no return values :

When a function has no arguments it doesnot receive any value from calling function.

As a function doesnot return any value, the calling function doesnot receive any value from called function.

Example :

```
#include<stdio.h>
void is_prime(); // function declaration
void main()
{
    printf("Program to check a number is prime or not");
    is_prime(); // function call
}

void is_prime()  // function definition
{
    int n, i, flag=0;
    printf("Enter a positive integer");
    scanf("%d", &n);
```

Scanned by CamScanner

```
for(i=2; i< n/2 ; i++)
{   if (n % i == 0)
    {   flag = 1;
    }
}
if (flag == 1)
    printf ("%d is not a prime number", n);
else
    printf ("%d is a prime number", n);
}
```

2. Functions with arguments and no return value :

Here calling function passes actual parameters to the called function. The actual and formal parameters should match in number, type and order.

The value of actual arguments are assigned to the formal arguments on one-to-one basis starting with the first argument.

· Example :

```
#include <stdio.h>
void simple_interest (float, int, int);
void main()
{   float p;
    int t, rate;
    printf ("Enter principal, time and rate");
    scanf ("%f %d %d", &p, &t, &r);
    simple_interest (p, t, r);
}
```

Scanned by CamScanner

```
simple_interest (float principal, int time, int r)
{
    float si;
    si = (principal * time * r) / 100;
    printf ("Simple interest = %f", si);
}
```

3. functions with no argument and return values:

In this case function will not take any arguments but returns a value to the calling function.

Example:

```
#include<stdio.h>
#include<math.h>
int square_root_sum();
int main()
{
    int num;
    num = square_root_sum();
    printf("\n Sum of square roots of given
            values is = %d", num);
    return 0;
}
int square_root_sum()
{
    int a, b;
    printf("Enter two values\n");
    scanf("%d %d", &a, &b);
    sum = sqrt(a) + sqrt(b);
    return sum;
}
```

Scanned by CamScanner

4. Function with arguments and return values:

```
#include <stdio.h>
double power(int, int);
void main()
{
    int x, y;
    printf("Enter x and y");
    scanf("%d %d", &x, &y);
    printf("%d to the power %d is %f\n",
                x, y, power(x, y));
}
double power(int x, int y)
{
    double p;
    p = 1.0;
    if(y >= 0)
        while(y--)
            p *= x;
    else
        while(y++)
            p /= x;
    return p;
}
```

5. Function returning multiple values:

In C if we want to get more information from a function we have to use arguments to send back to calling function.

The arguments that are used to send out information are called output parameters.

Scanned by CamScanner

Information send back through arguments can be achieved using address operator (&) and indirection operator (*).

```
#include<stdio.h>
void swap(int *x, int *y)
{
    int temp;
    temp = *x;
    *x = *y;
    *y = temp;
}
int main()
{
    int n1, n2;
    printf("Enter two values\n");
    scanf("%d %d", &n1, &n2);
    printf("Before swapping n1 is %d, n2 is %d",
            n1, n2);
    swap(&n1, &n2);
    printf("After swapping n1 = %d, n2 = %d", n1, n2);
    return 0;
}
```

Scanned by CamScanner

String Handling functions :

C supports a large number of string handling functions that can be used for manipulation of strings.

1. Strcat():

This function joins two strings together.
The general form is :

```
strcat ( string1, string2);
```

Here string2 is appended to string1 at the end by removing the null character at the end.

2. Strcmp():

This function compares two strings and has a value "0" if they are equal.

If they are not, it gives the numeric difference between the first non-matching character in the strings.

```
strcmp ( string1, string2);
```

3. Strcpy():

This function copies the content of one string to another.

```
strcpy ( city, "Delhi");
strcpy ( string1, string2);
```

Here the value of string2 is assigned to string1. So the size of string1 should be large enough to receive the content of string2.

4. Strlen():

The strlen function counts the no of character except '\0' and returns the value:

```
int n = strlen (String);
```

Scanned by CamScanner

5. strncpy():

This function is similar to strcpy(), except that at most n bytes of source string are copied.

If there is no NULL character among the first n character of source, the string copied to destination will not be NULL terminated.

If the length of source is less than n, strcpy() writes additional NULL character to destination.

```
strncpy(string1, string2, size);
```

6. strstr():

This function takes two strings s1 and s2 as argument and finds the first occurance of the substring s2 in the string s1.

The process of matching doesn't include null character, but the function stops there.

Example:

```
#include <string.h>
#include <stdio.h>
int main()
{   char *p;
    char s1[20], s2[20], s3[20];
    int n, l1, l2, l3;
    printf("\n Enter two strings \n");
    scanf("%s %s", s1, s2);
```

Scanned by CamScanner

```
x = strcmp ( s1, s2);
if (x!=0)
{
   printf ("\n Strings are not equal \n");
   strcat (s1, s2);
}
else
    printf ("\n Strings are equal \n");
strcpy (s3, s1);
 l1 = strlen (s1);
 l2 = strlen (s2);
 l3 = strlen (s3);
printf ("\n s1 = %s \t length = %d characters \n", s1, l1);
printf ("\n s2 = %s \t length = %d characters \n", s2, l2);
printf ("\n s3 = %s \t length = %d characters \n", s3, l3);
   p= strstr( s2, s3);
  if (p)
  {
     printf (" First occurance of string '%s' in '%s'
       is '%s' ", s3, s2, p);
  }
  else
     printf (" String not found \n");
return 0;
}
```

Scanned by CamScanner

Function Recursion :

The process in which a function calls itself directly or indirectly is called recursion and the corresponding function is called recursive function.

Example :

```
#include<stdio.h>
int fact(int);
int main()
{
    int factorial, n;
    printf("Enter the value of n");
    scanf("%d", &n);
    factorial = fact(n);
    printf("Factorial of n is %d", factorial);
    return 0;
}
int fact(int n)
{
    int f;
    if(n==1)
        return 1;
    else
        f = n * fact(n-1);
    return f;
}
```

When we write recursive functions we must have an if statement (terminate condition) to force the function to return without recursive call being executed.

Otherwise stack overflow problem will occur.

Scanned by CamScanner

Function Returning Pointers :

we know that a function can return a single value by its name or return multiple values through pointer parameters.

we also know pointers are data-types in C we can also force a function to return a pointer to the calling function.

Example

```
#include<stdio.h>
int *largler(int *, int *);
int main()
{
    int a=10;
    int b=20;
    int *p;
    p= largler(&a, &b);
    printf("%d", *p);
}
int *largler(int *x, int *y)
{
    if(*x > *y)
        return (x);
    else
        return (y);
}
```

The address returned must be address of a variable in the calling function. It is an error to return a pointer to a local variable in the called function.

Scanned by CamScanner

Pointer to Functions :

- Like variables, a function has a type and address location in the memory. Therefore it is possible to declare a pointer to a function.
- The general form of pointer to function is :

```
return_type (*function_pointer_name)(argument list);
```

Example :

```
int add(int x, int y); // function
int (*fptr)(int, int); // function pointer
fptr = &add;
```

```
#include<stdio.h>
int add(int,int);
int mul(int, int, int);
int main()
{
    int r1, r2, r3;
    int (*ptr1)(int, int);
    int (*ptr2)(int, int, int);
    r1 = add(10, 20);
    r2 = mul(2,3,4);
    printf("%d,%d\n", r1, r2);
    ptr1 = &add;
    r3 = ptr(30,50);
    printf("%d", r3);
    // ptr1 = &mul; // incompatible pointer assignment
    ptr2 = &mul;
    r3 = ptr2(5,10,20);
    printf("%d", r3);
    return 0;
}
```

Scanned by CamScanner

Command Line Arguments :

A command line argument is a parameter supplied to a program when the program is invoked.

We know that every-C program contains a "main()" function and program execution starts from main().

main() function can take two arguments called argc and argv and the information passed to the program through these arguments.

So these two arguments to main "argc" and "argv" are called command line arguments.

```
int main(int argc, char *argv[])
  {
    :
  }
```

- argc (ARGument Counter) is an integer variable that counts the number of arguments on the command line.
- argv (ARGument Vector) is an array of character pointers, listing all arguments.
- The array of pointers argv[0] to argv[argc-1] will contains the arguments as pointer to strings.
- argv[0] is the name of the program. After that till argv[argc-1] every element is command-line arguments.
- Command line arguments are supplied when program is invoked.

Scanned by CamScanner

Example :

```
#include <stdio.h>
int main ( int argc, char *argv[] )
{   int counter;
    printf (" Program name is: %s", argv[0]) ;
    if ( argc == 1)
       printf ("No extra argument is passed.\n");
    if ( argc >= 2)
    {
       printf ("Number of argument passed : %d", argc);
      for( counter =0; counter < argc ; counter ++)
          printf("\n argv[%d]: %s", counter,.
                                   argv [counter]);
    }
}
```

Dynamic Memory Allocation :

Dynamic memory allocation is a technique used in a situation where we deal with data, that are dynamic in nature.

Dynamic memory allocation allows us to deal with dynamic data structures which provides flexibility in adding, deleting or rearranging data items at run-time.

It allows us to allocate additional memory space or release unwanted space in runtime.

The process of allocating memory at runtime is known as dynamic memory allocation.

Scanned by CamScanner

* C Language provides the following four library functions for dynamic memory allocation:

malloc - Allocates requested size of bytes and returns a pointer to the first byte of the allocated space.

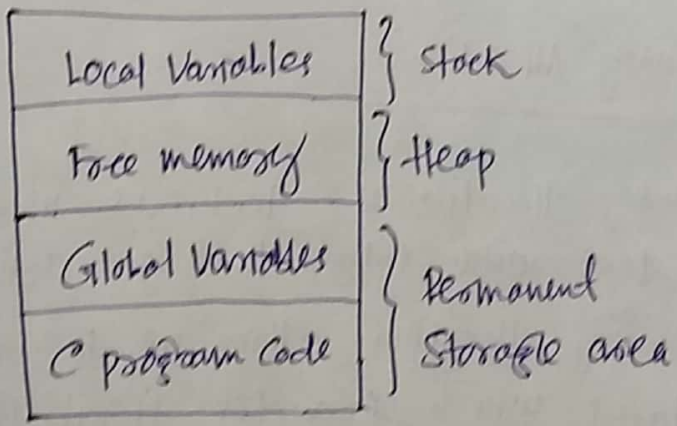
calloc - Allocates space for an array of elements initializes them to zero, return a pointer to the memory.

free - Frees the previously allocated space.

realloc - Modifies the size of previously allocated space.

Memory Allocation Process :

- When we run a C program the following memory allocation is associated with that C program.



- The program instructions and static variable and global variables are stored in a region called permanent storage area.
- Local variables and functions are created in stack area.
- The free memory region is called the heap. Dynamic memory allocation is performed in the heap area.

Scanned by CamScanner

malloc():

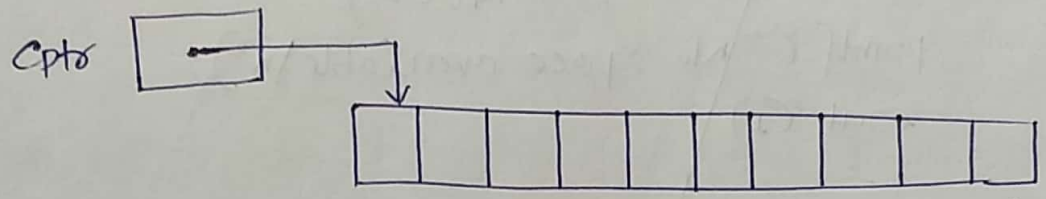
A block of memory may be allocated using the function malloc.

```
ptr = (cast-type) malloc (byte-size);
```

- malloc returns a pointer of type void. This means we can assign this to any type of pointer.

```
cptr = (char *) malloc (10);
```

- allocates 10 bytes of space for the pointer cptr of type char.



10 bytes of space.

calloc():

calloc allocates multiple blocks of storage area, each of the same size and then sets all bytes to zero.

The general form of calloc is :

```
ptr = (cast-type) calloc(n, element-size);
```

free():

Dynamically allocated memory are not released automatically. So we need a function to free these memory.
free() is a function which releases the allocated memory by malloc(), calloc() and realloc().

Syntax :

```
free(ptr);
```

Scanned by CamScanner

Example:

```
#include <stdio.h>
#include <stdlib.h>
int main()
{  int *pto, *list;
   int size;
   printf("What is the size of the table?");
   scanf("%d", &size);
   if ((list = (int *) malloc (sizeof(int) * size)) == NULL)
   {     printf("No space available\n");
         exit(1);
   }
   printf("\n Address of 1st byte of the list is %u", list);
   printf("\n Enter the values of the list\n");
   for (pto = list; pto < list + size; pto++)
         scanf("%d", pto);
   for (pto = list + size - 1; pto >= list; pto--)
         printf("%d is stored at address %u\n", *pto, pto);
   return 0;
}
```

realloc():

- Sometimes it happens that the previously allocated memory is not sufficient and we need additional space for some elements.
- It is also possible that the memory allocated is much larger than necessary and we want to reduce it.
- In both the cases we need to change the memory size of already allocated with the help of realloc() function.

Scanned by CamScanner

ptr = malloc (size);

ptr = realloc (ptr, newsize);

If realloc() is not able to find additional space in the same region, it will create an entire new memory area and move the content of ~~old~~ old block into the new block.

If function is unsuccessful in locating additional space, it returns a NULL pointer and the original block is freed (lost).

Example:

```
#include <stdio.h>
#include <stdlib.h>
int main()
{   char *buffer;
    if ((buffer = (char *) malloc(10)) == NULL)
    {       printf ("Malloc failed.\n");
            exit(1);
    }
    strcpy (buffer, "Balasore");
    printf ("Buffer contains : %s\n", buffer);
    if ((buffer = (char *)realloc(buffer, 15)) == NULL)
    {   printf("Reallocation failed.\n");
        exit(1);
    }
    printf ("Buffer contains : %s\n", buffer);
    strcpy (buffer, "Bhubaneswar");
    printf ("\n Buffer now contains : %s\n", buffer);
    free (buffer);
}   return 0;
```

Scanned by CamScanner